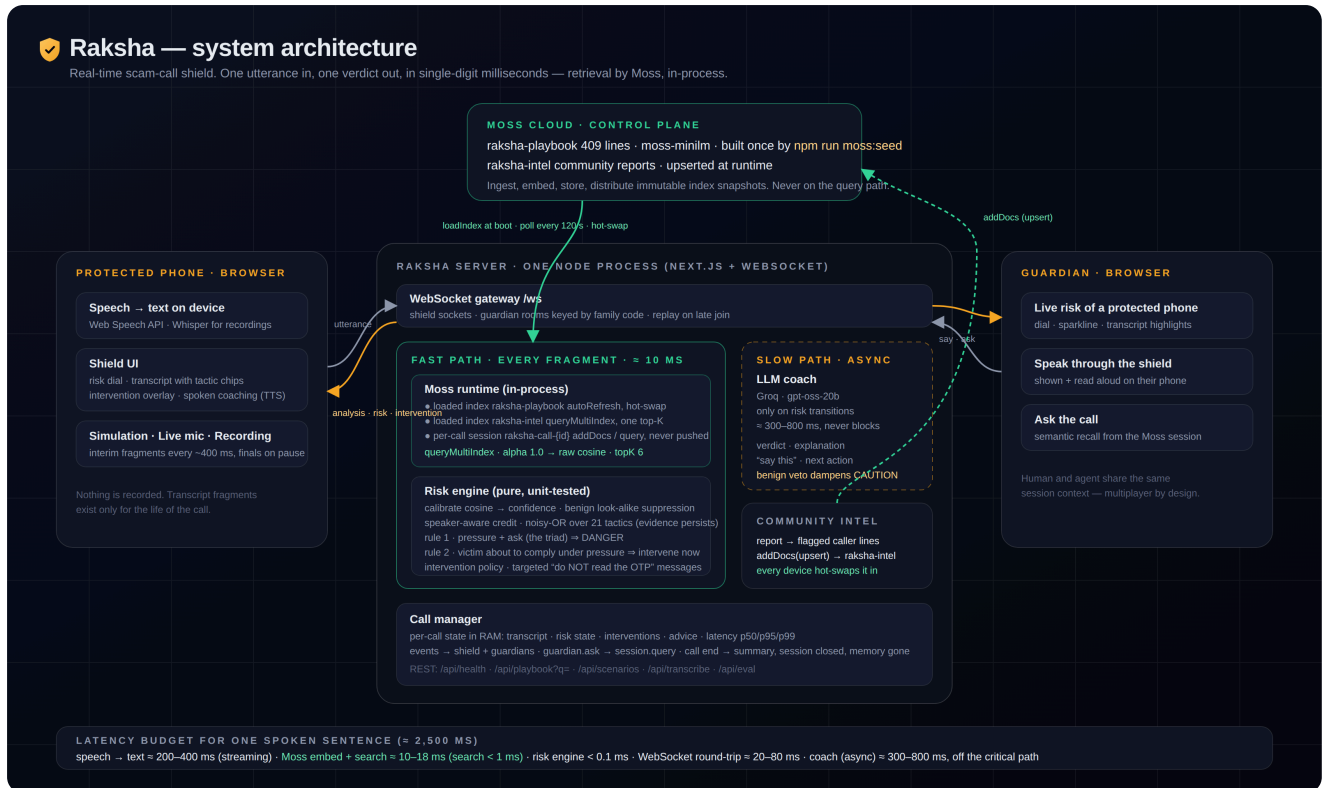


Raksha — Architecture

One utterance in, one verdict out, in single-digit milliseconds. This document explains how the pieces fit, why the retrieval layer sits where it does, and what the latency budget looks like.



1. The shape of the problem

A phone scam is a *conversation with a script*. The words change per family (digital arrest, bank OTP, courier parcel, deepfake CFO...) but the arc is constant: **pretext** → **pressure** → **isolation** → **the ask**. The victim is put into an emotional state where judgement shuts down (AARP calls it “the ether”), and the whole con is won or lost in the few seconds around the ask: “*read me the six digits, quickly.*”

That fixes three design constraints:

1. **Every fragment must be checked, not every sentence.** By the time a sentence has ended, the OTP may have been read out. We analyse interim speech-recognition fragments every ~400 ms as well as final segments.
2. **The check must cost nothing.** Hundreds of fragments per call, thousands of concurrent calls: a per-fragment cloud round-trip or LLM call is both too slow and too expensive. Local Moss queries are unmetered.
3. **The verdict must be explainable to a frightened 70-year-old, in one sentence, with one thing to say.**

Hence a **fast path** (Moss + a small deterministic risk engine, on every fragment) and a **slow path** (an LLM coach, only on risk transitions).

2. Components

Component	Where	Responsibility
Speech → text	Browser (Web Speech API), or Groq Whisper for recordings	Streams interim + final fragments. Nothing is stored.
Shield UI	Browser (/shield)	Risk dial, transcript with tactic chips, playbook matches, coach card, full-screen intervention with spoken coaching. Modes: simulation (scripted call with two TTS voices), live microphone, uploaded recording.
WebSocket gateway	server/ws.ts	One socket per protected phone or guardian. Routes utterances in, fans engine events out. Guardian rooms keyed by family code, with state replay on late join.
Call manager	src/lib/engine/calls.ts	Per-call state in RAM: transcript, RiskState , interventions, coach advice, latency percentiles. Writes each turn into the shared memory session tagged with the call id, and deletes them at call end.
Moss runtime	src/lib/moss/runtime.ts	Loads raksha-playbook (and raksha-intel when it exists) into the process at boot with autoRefresh ; opens one SessionIndex for call memory; answers every query in-process.
Retriever abstraction	src/lib/engine/retriever.ts	Retriever interface with two implementations: MossRetriever (production) and MockRetriever (TF-IDF cosine, zero credentials, used by CI).
Risk engine	src/lib/engine/risk.ts	Pure functions, unit-tested. Turns retrieval hits into a 0–100 score and a level (safe / caution / danger).
LLM coach	src/lib/llm/coach.ts	OpenAI-compatible chat call (Groq by default) returning {verdict, explanation, say_this, action} . Template fallback when no key is configured.
Community intel	src/lib/moss/intel.ts	On consent, upserts a call's flagged caller lines into raksha-intel ; every running instance hot-swaps the new version in.
Eval harness	scripts/eval.ts	Replays nine scripted calls through the real engine and writes docs/eval/REPORT.md .

Everything runs in **one Node process** (custom Next.js server + ws). That is deliberate: the Moss runtime, the loaded indexes and the per-call sessions must live in the same memory as the WebSocket handlers. It also means one container, one URL, no infrastructure.

3. The life of one utterance

```
sequenceDiagram
    autonumber
    participant P as Protected phone (browser)
    participant G as WS gateway
    participant C as Call manager
    participant M as Moss runtime (in-process)
    participant R as Risk engine
    participant L as LLM coach (async)
    participant D as Guardian (browser)

    P->>G: utterance {text, speaker, final:false} (interim, every ~400 ms)
```

```

G->>C: analyze(callId, fragment)
C->>M: query(playbook [+ intel], text, {topK:6, alpha:1.0})
M-->>C: matches + timeTakenInMs (≈ 2–5 ms)
C->>R: analyzeUtterance(prevState, utterance, matches)
R-->>C: RiskState (score, level, tactics, family, reasons)
alt interim crosses into DANGER
  C-->>P: intervention (overlay + spoken "STOP")
  C-->>D: intervention
end
P->>G: utterance {final:true}
G->>C: analyze(...) (same path)
C->>M: memory.addDocs([turn tagged callId]) (call memory, local)
C-->>P: analysis + risk + latency stats
C-->>D: analysis + risk
opt level changed, or 5 utterances since last coach
  C->>L: coach({transcript, risk, topMatches})
  L-->>C: {verdict, explanation, say_this, action} (≈ 300–800 ms)
  C->>R: applyCoachVerdict() (benign veto dampens CAUTION only)
  C-->>P: coach
  C-->>D: coach
end
D->>G: guardian.ask("what did they ask for?")
G->>C: ask(callId, question)
C->>M: memory.query(question, {topK:4, filter: callId})
M-->>D: top turns (≈ ms)

```

4. The risk model

The engine is intentionally small and inspectable (see `src/lib/engine/risk.ts`, 100% covered by `tests/unit/risk.test.ts`).

Calibration. Moss is queried with `alpha: 1.0` so scores are raw cosine similarities and comparable across queries. A score is mapped to a confidence with a linear ramp between a floor (noise) and a ceiling (near-paraphrase): `RAKSHA_SCORE_FLOOR / RAKSHA_SCORE_CEIL`, tuned by the eval harness.

Crediting. For one fragment we take the top hits, apply benign suppression (if a *legitimate look-alike* line scores as high as the best tactic line, the fragment is ignored — this is how “*we will never ask for your OTP*” stays quiet), apply a rank discount (the top hit is what the fragment *is*; the rest is what it *resembles*), and make it speaker-aware: the protected person's own words can only ever be evidence of **compliance** (reading an OTP, agreeing to transfer), never of the caller's pressure tactics — and vice-versa.

Accumulating. Each of 21 tactics keeps the best confidence seen in the call (evidence persists; a scam does not un-happen because the caller went quiet). The base score is a noisy-OR:

$$\text{score} = 100 \cdot (1 - \prod_t (1 - w_t \cdot \text{cap} \cdot \text{conf}_t))$$

so one strong tactic alone tops out around 30 (a bank *does* say “this is the bank”), while several distinct tactics compound quickly. Two hard rules mirror how scams actually end:

- **The triad.** A *pressure* tactic (authority, fear, urgency, secrecy, isolation, legal threat, hold-the-line...) plus an *ask* (OTP, payment, remote access, personal info) ⇒ at least **DANGER**.

- **Victim compliance.** If the person starts reading out a code or says "I'm transferring now" while any pressure tactic is active $\Rightarrow$ score ≥ 85 and an immediate, targeted intervention ("Do NOT read out the OTP.").

Levels: safe $< 25 \leq$ caution $< 60 \leq$ danger. The dominant scam family is the one with the largest confidence mass, with hysteresis so the label does not flip between sibling scripts (courier parcel $\rightarrow$ digital arrest) every turn.

Slow path veto. A confident *benign* verdict from the coach halves a CAUTION score. It never overrides DANGER: by then the triad has fired and the cost of a miss is someone's savings.

5. Where Moss does real work

Capability	How Raksha uses it	Why it matters
Loaded cloud index (<code>loadIndex</code> , <code>query</code>)	<code>raksha-playbook</code> (409 lines) loaded at boot, queried for every fragment with raw cosine scores.	The entire hot path is in-process: no vector DB, no network; ≈ 10 ms end-to-end including embedding.
Auto-refresh with hot-swap (<code>autoRefresh</code> , <code>pollingIntervalInSeconds</code>)	Both indexes poll every 120 s; newer versions swap in with zero query downtime.	New scam variants reach every running shield without a redeploy.
Sessions (<code>client.session</code> , <code>addDocs</code> , <code>query</code> with a metadata filter, <code>deleteDocs</code>)	One <code>SessionIndex</code> per process holds every live call's turns, each tagged <code>callId</code> ; the guardian's "ask the call" is a filtered semantic query over it, and a call's turns are deleted when it ends.	Live-call context with no persistence, at the cost of one embedding-model instance per process instead of one per call (opening a session costs ~ 2 s of CPU).
Multi-index search (<code>queryMultiIndex</code>)	Playbook + community intel searched in one call for a single global top-K.	Curated and crowd-sourced knowledge without merging indexes.
Metadata	Every line carries <code>family</code> , <code>tactics</code> , <code>severity</code> , <code>kind</code> (tactic / benign), <code>stage</code> , <code>region</code> .	The engine reasons over tactics, not raw text; benign look-alikes live in the same index.
Server-side embedding on ingest (<code>createIndex</code> , <code>addDocs</code> , <code>upsert</code>)	Seeding and community reports embed in Moss Cloud.	The API routes never need the model in memory.

Not used, on purpose: `pushIndex` (call memory must not persist by default), and the browser/WASM SDK for the hot path (its current release cannot use a delegated authenticator, so the project key would have to ship to the browser; the interface in `retriever.ts` is designed so the hot path can move on-device the day that lands).

6. Latency budget

For one spoken sentence (~ 2.5 s at conversational pace):

Stage	Typical	On the critical path?
Speech $\rightarrow$ text (streaming interim)	200–400 ms	yes (platform-bound)
WebSocket hop	20–80 ms	yes

Stage	Typical	On the critical path?
Moss embed + search	≈ 10–18 ms on a shared 4-vCPU container (search itself < 1 ms)	yes
Risk engine	< 0.1 ms	yes
LLM coach	300–800 ms	no — async, on transitions only

The `Latency lab` page measures the Moss numbers live against the running instance; `docs/eval/REPORT.md` records them for the committed evaluation.

6b. Scaling beyond one container

The single-process design is a deployment convenience, not an architectural limit:

- **Stateless by construction.** A container holds only (a) the loaded playbook/intel indexes, which every container loads identically from Moss Cloud, (b) one memory session, and (c) the state of the calls whose WebSockets it currently serves. There is no shared database to contend on.
- **Horizontal scaling** is therefore *N identical containers behind a WebSocket-aware load balancer with connection affinity*. A call lives entirely on the container that accepted its socket (its memory turns included), so nothing needs to be sharded.
- **Guardian rooms** are the one cross-container concern: a guardian's socket may land on a different container than the protected phone's. Roadmap: publish call events to a pub/sub channel (Redis or NATS) keyed by family code; each container subscribes for the codes it serves. Until then, affinity by family code (hash the code in the LB) keeps both sockets on one container.
- **Capacity, measured** (`npm run eval:load` , client and server on one 4-vCPU container, each call sending one fragment every ~3 s like a real conversation; the process has a single embedding executor, so this is the honest ceiling per container):

Concurrent calls (1 fragment / 3 s each)	Fragments/s	Server analysis p50 / p95 / p99 (ms)	Round-trip p50 / p95 (ms)	Errors
50	14.8	11.9 / 36.7 / 57.6	13.5 / 43.7	0
100	29.2	12.8 / 58.0 / 97.6	18.1 / 87.9	0

Memory is ~300 MB base plus a few KB per call. Beyond ~100 concurrent calls per container, add containers. An earlier design opened a Moss session per call and collapsed at 25 calls (p50 1.4 s) because each session loads its own model instance; the shared, call-id-filtered session fixed it.

- **Failure isolation.** A crash takes down only the calls on that container; clients reconnect (exponential back-off in `useRakshaSocket`) and start a fresh call. Health checks (`/api/health`) gate traffic until the index is loaded.

6c. Community intel: the pipeline in detail

Implemented in `src/lib/moss/intel.ts` and wired to the `call.report` WebSocket message:

1. **Consent.** Reporting is a deliberate tap on the post-call card, never automatic.

2. **Selection.** Only the *caller's* lines that were credited with a tactic are eligible (never the protected person's words); lines under 4 words are dropped.
3. **Abuse controls.** One report per call, a global budget per hour (`RAKSHA_INTEL_REPORTS_PER_HOUR` , default 60), and exact-duplicate suppression after normalisation, so a hostile client cannot flood the index.
4. **Write.** Lines are upserted into `raksha-intel` with metadata `{family, tactics, severity: 4, kind: tactic, source: community, reportedAt}`; Moss Cloud embeds them and publishes a new immutable index version.
5. **Propagation.** Every running container has `raksha-intel` loaded with `autoRefresh` (poll interval `MOSS_REFRESH_SECONDS` , default 120 s). When a newer version is detected it is hot-swapped atomically, so propagation latency is bounded by the poll interval plus the build time (typically under three minutes end to end).
6. **Containment.** Community lines carry `source: community` and a fixed severity, and they share top-K slots with the curated playbook; a poisoned line can add at most one credited tactic per fragment and can never suppress a benign look-alike. Roadmap: a moderation queue and per-reporter reputation before promotion into the curated playbook.

7. Deployment

- **Image:** `Dockerfile` (multi-stage, Node 22, non-root, health-check). `npm run build` produces the Next.js build and `dist/server.mjs` (esbuild bundle of the custom server).
- **Host:** any Docker host. The reference deployment is Google Cloud Run (`deploy/gcloud.sh` : Cloud Build + Cloud Run, session affinity for WebSockets, 1 vCPU / 1 GiB, asia-south1). `keepalive.yml` pings `/api/health` every 10 minutes so judges never hit a cold start.
- **State:** none outside the process except the Moss Cloud indexes. `MOSS_MODEL_CACHE_DIR` keeps the embedding model on the container's disk between restarts.
- **Config:** see `.env.example` . Without Moss credentials the app runs on the offline lexical fallback (so CI and forks work); without a Groq key the coach uses templates.
- **Degradation:** if Moss Cloud is unreachable at boot (network, credit limit, revoked key) the server starts on the offline retriever, reports the reason in `/api/health` and the shield header, and retries Moss every two minutes, swapping the real runtime in without a restart. Loaded indexes are cached on disk (`MOSS_CACHE_PATH`) so a restart only checks the version instead of re-downloading.

8. Security & privacy

- **Speech-to-text** happens in the browser via the Web Speech API. Chrome and Edge send audio to the vendor's speech service; Safari transcribes on-device. Raksha's server never receives audio in live mode, only text fragments, which it holds in RAM for the duration of the call and discards at call end. Roadmap: an on-device Whisper build (WebGPU) for browsers, and platform STT in the mobile app, so no audio leaves the phone at all.

- Recordings uploaded in "Recording" mode are streamed to Whisper on Groq for transcription (Groq's API does not retain audio) and are not stored by Raksha.
- The LLM coach receives only the recent transcript text and the risk summary, never audio, and only on risk transitions.
- Community reporting is opt-in per call and shares only the *caller's* flagged lines, never the protected person's words.
- The Moss project key lives on the server only. Family codes are capability tokens with no personal data behind them.
- No accounts, no cookies, no analytics.

8b. Privacy

See [PRIVACY.md](#) for the full data-handling table and threat model.

9. Repository map

```
server/           custom Next.js + WebSocket server (ws.ts routes, index.ts boot)
src/app/          pages: / shield guardian playbook lab, and /api/* route handlers
src/lib/engine/   types, risk engine, call manager, retriever interface, mock
retriever, latency
src/lib/moss/     Moss runtime (loading, sessions), MossRetriever, community intel
src/lib/llm/      provider-agnostic chat client, coach prompts
src/lib/data/     playbook codec, family + tactic catalogue
data/            playbook.json (409 lines), transcripts/ (9 scenarios)
scripts/         seed-moss, moss-status, eval, build-server
tests/           unit (vitest) and e2e (Playwright)
docs/            PRD, this document, eval report, research, video script, deck
```